

QUDA MultiGrid 粗层 $X/Y/\hat{Y}$ 、Clover-like dslash 与矩阵顺序

从聚合基、Galerkin 投影到递归求解的完全构建文档

PyQCU 工程分析记录

2026 年 9 月 13 日

摘要

本文是对 `refer/git-rep/quda` 中 Wilson/Clover MultiGrid 粗层实现的源码级补充解析，专门说明 onsite 矩阵 X 、方向链接 Y 、预条件链接 $\hat{Y}$ 的数学定义、存储位置、生成次序、Clover-like dslash 的真实行为以及矩阵左右顺序。QUDA 源码是第一证据，PyQCU strict full-coarse 实现是交叉验证。本文把 full operator、Clover-PC、coarse-PC 和 parity Schur 分开，最后以一个连续的超长单列表格给出从 null vector 到 $\hat{Y}$ 、MATPC 与验证的整条算法。

目录

1	范围、证据与六条结论	1
2	自由度、几何和量纲	2
3	聚合基、限制延拓和 Galerkin	3
4	field 分配、入口和 setup 工作区	3
5	X/Y 的数学构造	3
6	storage ABI、dslash 和 $\hat{Y}$ 次序	4
7	三种 dslash 语义及 Schur	4
8	一段贯穿全流程的超长公式型伪代码	5
9	递归、PyQCU 对照和验证	7
10	源码索引和最终不变量	8

1 范围、证据与六条结论

本文只研究给定 fine Wilson/Clover operator、聚合 null vectors 和 coarse geometry 后, QUDA 如何生成并应用

$$\{X_\ell, Y_{\ell,\mu}^f, Y_{\ell,\mu}^b, X_\ell^{-1}, \hat{Y}_{\ell,\mu}^f, \hat{Y}_{\ell,\mu}^b\}.$$

“构建”包括 field 分配、parity/方向映射、局部矩阵乘、atomic 累加、固定点缩放和 ghost exchange; “应用”包括 raw dslash、onsite block、preconditioned hopping、MATPC、prepare/reconstruct 及验证。

表 1: 证据分级

等级	材料	用法
源码事实	QUDA QUDA coarse operator、dslash 和 preconditioned kernel	描述真实参数、分支、存储和生命周期。
数学等价	$R = P^\dagger$ 、 $D_{\ell+1} = R A_\ell^{\text{eff}} P$ 、Schur 消元、 $\hat{Y}$ 定义	解释源码次序并做维数、共轭和量纲检查。
交叉实现	pyqcu/tools/_strict_gale rkin.py、pyqcu/solver/_qu da_multigrid.py、cpp/cuda/qcu/src/apply_multigrid_strict.cu	核对 full coarse、compact parity 和 QCU asset ABI。
未验证	尚未运行的 GPU 逐元素对照	明确标出, 不能用推导代替数值证据。

结论先写清楚。 X 是 coarse site 上的 dense $E \times E$ 矩阵, 不是 $SU(3)$; $Y_\mu^f(X)$ 存在源点 X , 乘 $z(X + \hat{\mu})$; $Y_\mu^b(X - \hat{\mu})$ 存在 $X - \hat{\mu}$, action 读取其 dagger。raw action 为

$$(D_c z)(X) = X(X)z(X) + \sum_\mu \{Y_\mu^f(X)z(X + \hat{\mu}) + Y_\mu^b(X - \hat{\mu})^\dagger z(X - \hat{\mu})\}.$$

预条件链接必为

$$\hat{Y}_\mu^f(X) = X^{-1}(X)Y_\mu^f(X), \quad \hat{Y}_\mu^b(X - \hat{\mu}) = Y_\mu^b(X - \hat{\mu})X^{-\dagger}(X).$$

`DiracCoarsePC::Dslash` 只应用 $\hat{Y}$ hopping; 它传入 X 但关闭 onsite 分支。Clover-PC 和 coarse-PC 需要双向链接; 普通 Wilson-like setup 可按 spin projector 规则由一侧 reverse。

参考源: docs/report_multigrid_quda_pyqcu_20260909.tex:547--691 是原报告对应章节; 本文逐项展开其结论。

2 自由度、几何和量纲

fine lattice 为 $\Lambda_f = (L_x, L_y, L_z, L_t)$, Wilson/Clover 每站点自由度为

$$e = N_s^f N_c^f = 4 \cdot 3 = 12.$$

block $b = (b_x, b_y, b_z, b_t)$ 给出

$$X_\mu = \lfloor x_\mu/b_\mu \rfloor, \quad \Lambda_c = (L_x/b_x, L_y/b_y, L_z/b_z, L_t/b_t).$$

coarse spin 为 $N_s^c = 2$, null-vector 数为 n_v , 故 $E = N_s^c n_v$, $X, Y, \hat{Y} \in \mathbb{C}^{E \times E}$ 。 $n_v = 12$ 时 $E = 24$, 不是把 fine spin 直接删成 3×3 link。

定义 checkerboard

$$p(x) = (x + y + z + t) \bmod 2, \quad p = 0 \text{ (even)}, \quad p = 1 \text{ (odd)}.$$

hopping 改变 parity, onsite 不改变。full coarse assets 保留 Λ_c 全格; MATPC 输入才压缩一半。固定前三维时 parity compact 的时间坐标为

$$t_{\text{full}} = 2t_{\text{half}} + [(p - x - y - z) \bmod 2].$$

参考源: `cpp/cuda/qcu/src/apply_multigrid_strict.cu:108--159` 定义 full/half site; :230-282 使用 parity decode; `pyqcu/tools/_strict_galerkin.py:378--405` 检查周期 checkerboard 的偶数 extent。

3 聚合基、限制延拓和 Galerkin

令 $V_{s_f c_f, s_c j}(x)$ 为 aggregate 内的正交基:

$$(P\phi)_{s_f c_f}(x) = \sum_{s_c, j} V_{s_f c_f, s_c j}(x) \phi_{s_c j}(X(x)), \quad (1)$$

$$(R\psi)_{s_c j}(X) = \sum_{x \in A_X} \sum_{s_f, c_f} V_{s_f c_f, s_c j}(x)^* \psi_{s_f c_f}(x), \quad R = P^\dagger. \quad (2)$$

局部 CGS 给出

$$V_X^\dagger V_X = I_E, \quad P^\dagger P = I_{\text{coarse}}, \quad PP^\dagger \neq I_{\text{fine}}.$$

任意有效 action 的下一层为

$$D_{\ell+1} = R_\ell A_\ell^{\text{eff}} P_\ell, \quad A_\ell^{\text{eff}} = D_\ell \text{ (raw)}, \quad A_\ell^{\text{eff}} = X_\ell^{-1} D_\ell \text{ (strict PC)}.$$

one-hop support 只允许 $\mathcal{N}(X) = \{X, X \pm \hat{\mu}\}$; 发现更远 displacement 就不能静默压缩成 X/Y 。

参考源: QUDA transfer: `refer/git-rep/quda/lib/transfer.cpp:152--163`; PyQCU batched support: `pyqcu/tools/_strict_galerkin.py:489--720, 678--699`。

4 field 分配、人口和 setup 工作区

`DiracClover::createCoarseOp` 调 `CoarseOp`, `DiracCloverPC::createCoarseOp` 传入 PC 类型; 源码在 `refer/git-rep/quda/lib/dirac_clover.cpp:95--103, 263--271`。coarse field 分配在 `DiracCoarse::createY`:

$$\text{nColor} = N_s^c n_v, \quad \text{link_type} = \text{COARSE_LINKS}, \quad \text{siteSubset} = \text{FULL}, \quad \text{nFace} = 1.$$

`createYhat` 再分配 $\hat{Y}$ 和无 ghost 的 X^{-1} , 见 `refer/git-rep/quda/lib/dirac_coarse.cpp:121--220`。GPU native order 与 CPU QDP order 分开; Y/X order 不一致会在:474 或:644 直接报错。

`calculateY` 计算 aggregation、spin block、face 数和 `bidirectional\_links`, 见 `refer/git-rep/quda/lib/coarse_op.cuh:974--1058`。工作区是

$$AV = C^{-1}V \quad (\text{Clover-PC}), \quad UV = UV \quad \text{或} \quad U AV.$$

先生成 UV , 再做 $(AV)^\dagger UV$ 的 contraction。Clover inverse 的 packed chiral kernel 为 `refer/git-rep/quda/include/kernels/coarse_op_kernel.cuh:693--760`; 非预条件 path 令 $AV = V$ 。

5 X/Y 的数学构造

forward link 为

$$Y_\mu^f(X) = -\kappa \sum_{x \in \mathcal{A}_X} (AV_X(x))^\dagger U_\mu(x) V_{X+\hat{\mu}}(x + \hat{\mu}).$$

backward storage block 为

$$Y_\mu^b(X - \hat{\mu}) = -\kappa \sum_{x \in \mathcal{A}_X} V_X(x)^\dagger U_\mu^\dagger(x - \hat{\mu}) AV_{X-\hat{\mu}}(x - \hat{\mu}).$$

实际代码可把 $-\kappa$ 延后到 `dslash` 的 `dim\_collapse`, 所以比较 buffer 时必须记录系数是否已吸收。`computeUV` 以 `mma\_nn` 做 UV , `multiplyVUV` 以 `mma\_tn` 做共轭转置左乘, 见 `refer/git-rep/quda/include/kernels/coarse_op_kernel.cuh:255--315, 1157--1226`。多个 fine site 和 basis pair 并发写同一粗块, 故使用 `complex atomic add`。

full Clover onsite 是

$$X_C(X) = \sum_{x \in \mathcal{A}_X} V_X(x)^\dagger C(x) V_X(x),$$

`compute\_coarse\_clover` 在 `refer/git-rep/quda/include/kernels/coarse_op_kernel.cuh:1768--1836`; Wilson-like 分支还执行 $X \leftarrow X + I_E$, 见:1878–1898。coarse-to-coarse 重新投影当前层 X_ℓ , 不是读取 fine C 。Clover-like 表示 dense local block 和 inverse, 不表示 fine 的 packed Clover tensor。

Clover-PC、Coarse-PC、twisted-PC 或 `need\_bidirectional` 保存两向; 普通 operator 可调用 `reverse`。diagonal coarse-spin block 保持符号, off-diagonal 改变符号, 见 `refer/git-rep/quda/include/kernels/coarse_op_kernel.cuh:1840--1875`。

6 storage ABI、`dslash` 和 $\hat{Y}$ 次序

QUDA coarse kernel 注释给出

$$/out(x) = \sum_{\mu} Y_{-\mu}(x) in(x + \mu) + Y_{\mu}^\dagger(x - \mu) in(x - \mu).$$

forward 读源点 forward slot; backward 用 $x - \mu$ 的 storage block 并取 dagger, 源码见 `refer/git-rep/quda/include/kernels/dslash_coarse.cuh:126--249`。

令 action 目标点为 q 。backward 项为

$$Y_\mu^b(q - \hat{\mu})^\dagger z(q - \hat{\mu}).$$

左预条件后的项必须满足

$$X^{-1}(q)Y_\mu^b(q - \hat{\mu})^\dagger = \left[Y_\mu^b(q - \hat{\mu})X^{-\dagger}(q) \right]^\dagger,$$

故

$$\boxed{\hat{Y}_\mu^b(q - \hat{\mu}) = Y_\mu^b(q - \hat{\mu})X^{-\dagger}(q)}.$$

错误的 $X^{-1}(q - \hat{\mu})Y_\mu^b(q - \hat{\mu})$ 同时错 site、错侧；非对易 dense block 会暴露该错误。

calculateYhat 的真实顺序是：batch invert X ；双向交换 Y ghost；backward 计算 $YX^{-\dagger}$ ；forward 计算 $X^{-1}Y$ ；注入 backward bulk；只交换 forward ghost。源码注释和实现为 refer/git-rep/quda/lib/coarse_op_preconditioned.in.cu:148--282。不变量为

$$X^{-1}X = I, \quad \hat{Y}^f = X^{-1}Y^f, \quad (\hat{Y}^b)^\dagger = X^{-1}(Y^b)^\dagger.$$

7 三种 dslash 语义及 Schur

定义 raw hopping

$$H_Y z(X) = \sum_\mu [Y_\mu^f(X)z(X + \hat{\mu}) + Y_\mu^b(X - \hat{\mu})^\dagger z(X - \hat{\mu})].$$

若 $-\kappa$ 未吸收进 link, $D_c z = Xz - \kappa H_Y z$ 。DiracCoarse::Dslash 传 clover=false; M 以 clover=true, dslash=true, xpay=true 组合, 见 refer/git-rep/quda/lib/dirac_coarse.cpp:387--433。onsite kernel 是 $out+ = X in$ 或 $X^\dagger in$, 见 refer/git-rep/quda/include/kernels/dslash_coarse.cuh:257--289。

DiracCoarsePC::Dslash 选择 $\hat{Y}$ 且 clover=false, 见 refer/git-rep/quda/lib/dirac_coarse.cpp:506--519; X 只供 prepare/reconstruct/raw path, 不能在 PC dslash 再乘一次。对 target parity:

$$u_q = \hat{H}_{qp}x_p, \quad v_p = \hat{H}_{pq}u_q, \quad M_p^c x_p = x_p - v_p.$$

:530-561 的 even-even/odd-odd 分支拒绝 full field; symmetric branch 两次 hop 后以 xpay(-1) 形成 $I - \hat{H}\hat{H}$ 。

按 parity 排列

$$D = \begin{pmatrix} A_p & D_{pq} \\ D_{qp} & A_q \end{pmatrix}, \quad x_q = A_q^{-1}(b_q - D_{qp}x_p),$$

$$S_p = A_p - D_{pq}A_q^{-1}D_{qp}, \quad M_p = A_p^{-1}S_p, \quad b_p^{\text{prep}} = A_p^{-1}(b_p - D_{pq}A_q^{-1}b_q).$$

full solution 的 coarse prepare/reconstruct 为

$$b_p^{\text{prep}} = X_p^{-1}b_p - (X_p^{-1}D_{pq})X_q^{-1}b_q, \quad x_q = X_q^{-1}b_q - (X_q^{-1}D_{qp})x_p.$$

源码先 inverse、再 raw dslash、最后 axpy, 见 refer/git-rep/quda/lib/dirac_coarse.cpp:570--625。

表 2: full、Clover-PC、coarse-PC 对照

输入	中间量	最终对象	双向
Clover full	$AV = V$	$Y = V^\dagger UV,$ $X = V^\dagger CV + I$ (按 convention)	可 reverse
Clover-PC	$AV = C^{-1}V$	Y 含 inverse 作用, MATPC 显式归一化	必须
coarse Dirac	当前层 X_ℓ, Y_ℓ	$X_{\ell+1} = V^\dagger X_\ell V$	传递
coarse PC	输入是 $\hat{Y}_\ell$	下一层 coarsen $\hat{Y}$, 不是 raw Y	必须

参考源: [refer/git-rep/quda/lib/dirac_coarse.cpp:628--647](#) 明确 coarse-PC coarsens $\hat{Y}$ rather than Y 。

8 一段贯穿全流程的超长公式型伪代码

以下一个连续 `longtable` 每行只表达一个动作、等式或分支, 行号不拆成独立算法。

表 3: 从 null vectors 到递归求解的连续算法

输入: $\Lambda_f, U_\mu, C, V, b, \kappa, m, \mu$, operator type, target parity p 。
检查: $e = 12$, $E = 2n_v$, block size 整除 fine extent, coarse extent 为正整数。
检查: periodic checkerboard 的每个非平凡 coarse extent 为偶数。
定义: $X(x) = \lfloor x/b \rfloor$, $\mathcal{A}_X = \{x : X(x) = X\}$ 。
正交: 每个 aggregate 内 null vectors 做两遍 block CGS。
不变量: $\ V_X^\dagger V_X - I_E\ \leq \varepsilon_{\text{orth}}$, 失败停止。
定义: $(P\phi)(x) = V_X \phi(X)$, $(R\psi)(X) = V_X^\dagger \psi _{\mathcal{A}_X}$ 。
Galerkin: $D_c = RA_{\text{eff}}P$; raw 取 $A_{\text{eff}} = D_f$, strict-PC 取 $A_{\text{eff}} = X_f^{-1}D_f$ 。
支持: 只保留 $\Delta = 0, \pm\hat{\mu}$, 更远项必须报错。
分配: full Y 八方向 slot、full X 、atomic Y/X 和工作区 UV/AV 。
分配: PC 再分配 full $\hat{Y}$ 与 scalar X^{-1} 。
精度: half/quarter 使用 single accumulator, 保存前估计 maximum 和 scale。
交换: 计算跨 rank 的 UV 前交换 V 的 one-face ghost。
PC 分支: $AV(x) \leftarrow C^{-1}(x)V(x)$; 非 PC 令 $AV \leftarrow V$ 。
方向: 对 $d = 0, 1, 2, 3$ 设置 FORWARDS。
forward UV: $UV_d(x) \leftarrow U_d(x)V(x + \hat{d})$ 。
forward contraction: $Y_d^f(X) \leftarrow (AV_X)^\dagger UV_d$ 。
forward coefficient: 若未吸收系数, $Y_d^f \leftarrow -\kappa Y_d^f$ 。
forward storage: 写入 source aggregate X 的 forward slot。

(续表：同一算法，行号连续)

backward UV: $UV_d^b(x) \leftarrow U_d^\dagger(x - \hat{d})AV(x - \hat{d})$ 。

backward contraction: $Y_d^b(X - \hat{d}) \leftarrow V_X^\dagger UV_d^b$ 。

backward storage: 写入 $X - \hat{d}$, action 端读取其 dagger。

双向: Clover-PC、Coarse-PC 或 `need\_bidirectional` 时保存两向。

reverse: 单向普通 operator 用 spin-projector 符号规则生成另一向。

atomic: 对 fine site、basis pair、parity 并发 complex atomic add。

edge: 同 aggregate 贡献进入 X , 跨 aggregate 贡献进入 Y 。

Clover onsite: $X_C \leftarrow V^\dagger CV$ 。

identity onsite: $X \leftarrow X + I_E$ 。

coarse onsite: $X_{\ell+1} \leftarrow V^\dagger X_\ell V$ 。

scale: 检查每个方向 maximum; 超出旧 scale 时重放 rescale。

convert: atomic fixed-point buffer 解码为应用 field。

raw action: $D_c z = Xz - \kappa \sum_d (Y_d^f z_{+d} + Y_d^{b\dagger} z_{-d})$ 。

raw forward: 读取 $Y_d^f(X)$ 和 $z(X + \hat{d})$ 。

raw backward: 读取 $Y_d^b(X - \hat{d})^\dagger$ 和 $z(X - \hat{d})$ 。

raw onsite: `applyClover` 计算 $out += X in$ 。

raw M: `clover=true, dslash=true, xpay=true`。

求逆: 对每个 coarse site batch invert X 。

Yhat forward: $\hat{Y}_d^f(X) \leftarrow X^{-1}(X)Y_d^f(X)$ 。

Yhat backward: $\hat{Y}_d^b(X - \hat{d}) \leftarrow Y_d^b(X - \hat{d})X^{-\dagger}(X)$ 。

不变量: $(\hat{Y}_d^b)^\dagger = X^{-1}(Y_d^b)^\dagger$ 。

ghost: backward inject 到上一节点, 再只 exchange forward。

PC hop: $\hat{H}z = \sum_d (\hat{Y}_d^f z_{+d} + \hat{Y}_d^{b\dagger} z_{-d})$ 。

MATPC 输入: 必须是 compact target parity; full field 直接拒绝。

MATPC 第一步: $u_q \leftarrow \hat{H}_{qp} x_p$ 。

MATPC 第二步: $v_p \leftarrow \hat{H}_{pq} u_q$ 。

MATPC 输出: $M_p^c x_p \leftarrow x_p - v_p$ 。

prepare: $b_p^{\text{prep}} \leftarrow X_p^{-1} b_p - (X_p^{-1} D_{pq}) X_q^{-1} b_q$ 。

reconstruct: $x_q \leftarrow X_q^{-1} b_q - (X_q^{-1} D_{qp}) x_p$ 。

递归: coarse-PC 下一层输入必须是 $\hat{Y}$, 不是 raw Y 。

资产: 缓存 $V, X, X^{-1}, Y^{f/b}, \hat{Y}^{f/b}$, parity 只是 view。

QCU ABI: strict transition 固定输出 $V/\text{raw } Y / \hat{Y} / (X, X^{-1})$ 。

QCU full hop: forward link 与 z_{+d} 、backward link dagger 与 z_{-d} 。

QCU parity hop: 先 decode full coordinate, 再取 compact neighbor。

Galerkin test: $\epsilon_G = \|D_c z - RD_f Pz\| / \max(1, \|RD_f Pz\|)$ 。

transfer test: $\epsilon_{PR} = |\langle Pz, f \rangle - \langle z, Rf \rangle| / \max(1, \cdot)$ 。

inverse test: $\epsilon_X = \|X^{-1} X - I\| / \max(1, \|I\|)$, 非正规 X 同测 XX^{-1} 。

direction test: unit basis 分别比较 forward/backward storage。

极限: $X = I \Rightarrow \hat{Y} = Y$; $Y = 0 \Rightarrow \text{hop 为零}$; $E = 1$ 检查 site/parity。

(续表：同一算法，行号连续)

非对易测试： $X = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ 、 $Y = \begin{pmatrix} 1 & 0 \\ 1 & 2 \end{pmatrix}$ 时 $X^{-1}Y \neq YX^{-1}$ 。

MPI：单 rank 过 bulk，再在 boundary 比较 ghost 前后的 action。

精度：double reference、single compute、half storage 分别报告 inverse/Galerkin/solve residual。

停止：support、transfer、inverse、storage、ghost、精度不变量全通过才进入下一层。

递归求解：以当前 coarse operator 为下一层 fine operator，重复 basis、Galerkin、 X/Y 、可选 $\hat{Y}$ 。

输出：field、geometry、 E 、precision、scale、parity metadata 和验证报告。

参考源：QUDA 源：refer/git-rep/quda/lib/coarse_op.cuh:974--1463、refer/git-rep/quda/lib/coarse_op_preconditioned.in.cu:148--282、refer/git-rep/quda/lib/dirac_coarse.cpp:506--647；PyQCU：pyqcu/tools/_strict_galerkin.py:334--720。

9 递归、PyQCU 对照和验证

每层资产为

$$\mathcal{G}_\ell = \{V_\ell, X_{\ell+1}, X_{\ell+1}^{-1}, Y_{\ell+1}^{\text{f/b}}, \hat{Y}_{\ell+1}^{\text{f/b}}\}.$$

V-cycle 递推为

$$r_\ell = b_\ell - A_\ell x_\ell, \quad r_{\ell+1} = R_\ell r_\ell, \quad e_{\ell+1} \simeq A_{\ell+1}^{-1} r_{\ell+1}, \quad x_\ell \leftarrow x_\ell + P_\ell e_{\ell+1}.$$

PyQCU strict 的 `\_strict\_v\_cycle` 在 `pyqcu/solver/_quda_multigrid.py:3005--3048` 先在 target parity 平滑，再恢复 full coarse rhs；diagnostics 的 Galerkin 对照为:3396-3402。QCU full kernel 在 `cpp/cuda/qcu/src/apply_multigrid_strict.cu:161--282` 与 QUDA storage 同构。

验证量为

$$\epsilon_{PR} = \frac{|\langle Pz, f \rangle - \langle z, Rf \rangle|}{\max(1, |\langle Pz, f \rangle|, |\langle z, Rf \rangle|)}, \quad \epsilon_X = \frac{\|X^{-1}X - I\|}{\max(1, \|I\|)}, \quad (3)$$

$$\epsilon_G = \frac{\|D_c z - RD_f Pz\|}{\max(1, \|RD_f Pz\|)}, \quad \epsilon_b = \frac{\|(\hat{Y}^b)^\dagger - X^{-1}(Y^b)^\dagger\|}{\max(1, \|X^{-1}(Y^b)^\dagger\|)}. \quad (4)$$

表 4: 极限测试与错误定位

情形	预期	定位
$X = I$	$X^{-1} = I, \hat{Y} = Y$	inverse、方向或 scale
$Y = 0$	只剩 onsite, PC hop 为零	clover flag 或 xpay
$E = 1$	左右乘退化为标量	site/parity 错位
单 block	off-block link 消失	aggregate edge
周期边界	零点 backward 读取末点	ghost inject/exchange
	storage	
非对易 X, Y	左右乘结果不同	$\hat{Y}$ 次序

情形	预期	定位
full MATPC input	拒绝或先 compact view	ABI 混用
half storage	解码矩阵接近 single reference	scale

GPU 逐元素回归必须固定 gauge、null vectors、geometry、precision 和 parity，同时报告 raw X/Y 、 X^{-1} 、 $\hat{Y}$ 、dslash、MATPC 和 Galerkin residual；solver residual 不能替代这些局部证据。本文生成过程未运行完整 GPU 逐元素回归，故该项为“未验证”。

10 源码索引和最终不变量

表 5: 直接引用源

源文件	内容
refer/git-rep/quda/lib/dirac_coarse.cpp:121--220	$Y/X/Yhat/Xinv$ 分配、geometry、order、ghost
refer/git-rep/quda/lib/dirac_coarse.cpp:420--478	raw M、prepare/reconstruct、coarse setup
refer/git-rep/quda/lib/dirac_coarse.cpp:506--647	PC dslash、MATPC、prepare/reconstruct、递归 $\hat{Y}$
refer/git-rep/quda/lib/coarse_ops.cuh:974--1463	UV、AV、双向、atomic、reverse、onsite、scale
refer/git-rep/quda/include/kernels/coarse_op_kernel.cuh:255--315, 619--760, 1157--1226	UV、 $C^{-1}V$ 、 $(AV)^\dagger UV$
refer/git-rep/quda/include/kernels/coarse_op_kernel.cuh:1608--1647, 1768--1898	edge、parity、Clover、identity
refer/git-rep/quda/include/kernels/dslash_coarse.cuh:126--289	forward/backward gather、dagger、onsite
refer/git-rep/quda/lib/coarse_ops_preconditioned.in.cu:148--282	batch inverse、 $YX^{-\dagger}$ 、 $X^{-1}Y$ 、ghost
pyqcu/tools/_strict_galerkin.py:334--720	strict assets、Galerkin、support、MATPC
pyqcu/solver/_quda_multigrid.py:3193--3402	asset export、V-cycle、diagnostics
cpp/cuda/qcu/src/apply_multigrid_strict.cu:161--282	QCU full/parity hop
docs/report_multigrid_quda_pyqcu_20260909.tex:399--691	原报告 Schur、P/R、Galerkin、 $X/Y/\hat{Y}$ 记号

1. 每层 $E = 2n_v$; $X, Y, \hat{Y}$ 是 dense coarse blocks, 不是 $SU(3)$ 。
2. raw backward 必须取 storage link dagger。
3. $\hat{Y}^f = X^{-1}Y^f$, $\hat{Y}^b = Y^bX^{-\dagger}$ 。
4. PC dslash 只做 $\hat{Y}$ hopping; MATPC 是 $I - \hat{H}\hat{H}$ 。
5. Clover-PC/coarse-PC 双向构造; 递归 coarse-PC coarsen $\hat{Y}$ 。
6. full assets 与 compact parity view 分离。
7. 验证必须覆盖 transfer、inverse、Galerkin、storage、ghost 和精度。

参考源：源码与公式解析已完成；完整 GPU 逐元素回归依赖本机 CUDA/QUDA 构建环境，本次未宣称已完成。